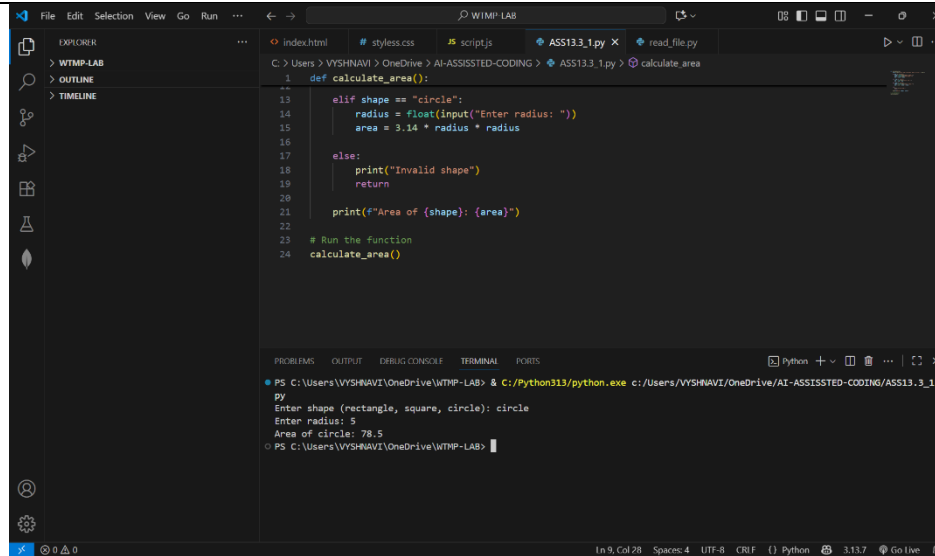


SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025-2026
Course Coordinator Name		Venkataramana Veeramsetty	
Instructor(s) Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr. J. Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S. Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch. Rajitha	
		Mr. M Prakash	
		Mr. B. Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
		NS_2 (Mounika)	
Course Code	24CS002PC215	Course Title	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week 7 - Wednesday	Time(s)	
Duration	2 Hours	Applicable to Batches	
Assignment Number: 13.3 (Present assignment number) / 24 (Total number of assignments)			
Q.No.	Question		Expected Time to complete
1	Lab 13 – Code Refactoring: Improving Legacy Code with AI Suggestions Lab Objectives To introduce the concept of code refactoring and why it matters (readability, maintainability, performance).		Week 5 - Monday

	• To practice using AI tools for identifying and suggesting improvements in legacy code. • To evaluate the before vs. after versions for clarity, performance, and correctness. • To reinforce responsible AI-assisted coding practices (avoiding over-reliance, validating outputs). Learning Outcomes After completing this lab, students will be able to: 1. Use AI to analyze and refactor poorly written Python code. 2. Improve code readability, efficiency, and error handling. 3. Document AI-suggested improvements through comments and explanations. 4. Apply refactoring strategies without changing functionality. 5. Critically reflect on AI's refactoring suggestions. Task Description #1 – Remove Repetition Task: Provide AI with the following redundant code and ask it to refactor Python Code <pre>def calculate_area(shape, x, y=0): if shape == "rectangle": return x * y elif shape == "square": return x * x elif shape == "circle": return 3.14 * x * x</pre> Expected Output • Refactored version with dictionary-based dispatch or separate functions. • Cleaner and modular design. PROMPT: Refactor the shape area calculator using dictionary-based dispatch or separate functions for cleaner, modular design. Ensure each shape's logic is isolated and easily extendable. CODE SCREENSHOT:	
--	--	--



```
def calculate_area():
    13     elif shape == "circle":
    14         radius = float(input("Enter radius: "))
    15         area = 3.14 * radius * radius
    16
    17     else:
    18         print("Invalid shape")
    19         return
    20
    21     print(f"Area of {shape}: {area}")
    22
    23     # Run the function
    24     calculate_area()
```

```
PS C:\Users\VYSHNAVI\OneDrive\WTHP-LAB> & c:/Python313/python.exe c:/Users/VYSHNAVI/OneDrive/AI-ASSISTED-CODING/ASS13.3.1.py
Enter shape (rectangle, square, circle): circle
Enter radius: 5
Area of circle: 78.5
PS C:\Users\VYSHNAVI\OneDrive\WTHP-LAB>
```

Observation:

The refactored code using dictionary-based dispatch is clean, modular, and easy to extend. Each shape's logic is separated into its own function, improving readability and reducing repetition. The use of a dispatch table simplifies control flow and makes the program scalable for adding more shapes. Overall, it's a well-structured and efficient design.

Task Description #2 – Error Handling in Legacy Code

Task: Legacy function without proper error handling

Python Code

```
def read_file(filename):
    f = open(filename, "r")
    data = f.read()
    f.close()
    return data
```

Expected Output:

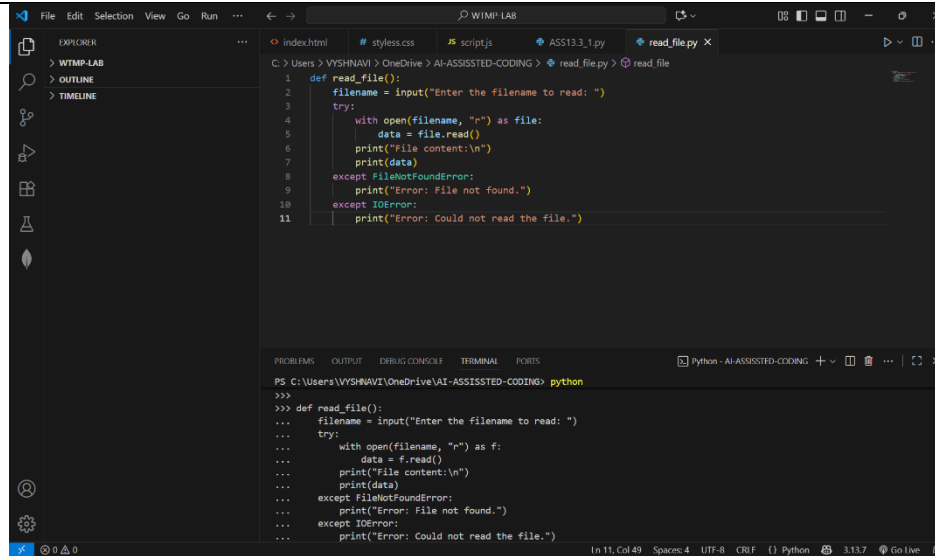
AI refactors with with open() and try-except:

PROMPT:

Write a Python program that asks the user for a filename and reads its contents.

Use with open() and try-except blocks to handle file errors gracefull

CODE SCREENSHOT:



```
1 def read_file():
2     filename = input("Enter the filename to read: ")
3     try:
4         with open(filename, "r") as file:
5             data = file.read()
6             print("File content:\n")
7             print(data)
8     except FileNotFoundError:
9         print("Error: File not found.")
10    except IOError:
11        print("Error: Could not read the file.")
```

```
>>> def read_file():
...     filename = input("Enter the filename to read: ")
...     try:
...         with open(filename, "r") as f:
...             data = f.read()
...             print("File content:\n")
...             print(data)
...     except FileNotFoundError:
...         print("Error: File not found.")
...     except IOError:
...         print("Error: Could not read the file.")
```

OBSERVATION:

This Python function `read_file()` prompts the user to enter a filename, then attempts to open and read the file using a `with open()` block. It handles errors gracefully using `try-except`, specifically catching `FileNotFoundError` and `IOError`, and prints appropriate messages if the file cannot be accessed

Task Description #3 – Complex Refactoring

Task: Provide this legacy class to AI for readability and modularity improvements:

Python Code

```
class Student:
    def __init__(self, n, a, m1, m2, m3):
        self.n = n
        self.a = a
        self.m1 = m1
        self.m2 = m2
        self.m3 = m3
    def details(self):
        print("Name:", self.n, "Age:", self.a)
    def total(self):
        return self.m1+self.m2+self.m3
```

Expected Output:

- AI improves naming (name, age, marks).

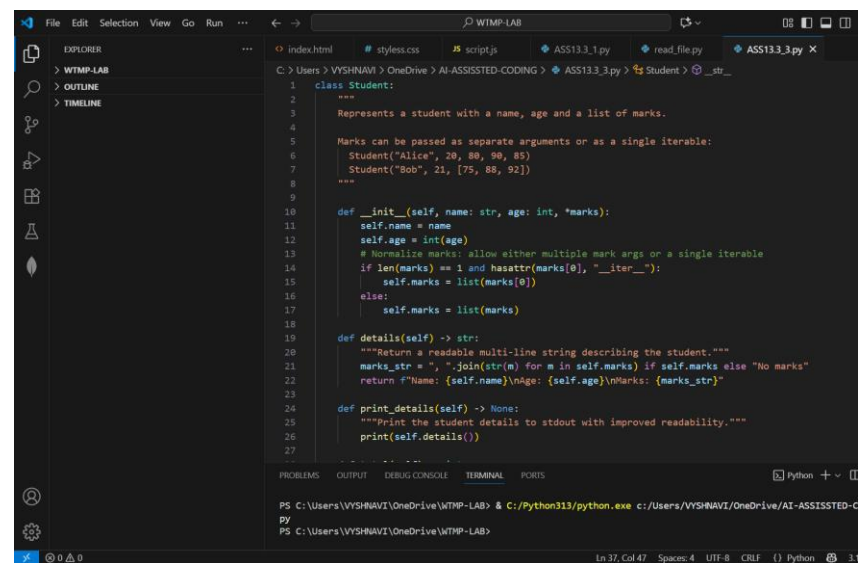
- Adds docstrings.
- Improves print readability.
- Possibly uses `sum(self.marks)` if marks stored in a list.

PROMPT:

Write a Python program using a class to represent a student with name, age, and marks in three subjects.

Accept user input to create a student object, display the student's details, and calculate the total marks.

CODE SCREENSHOT:



```

1 class Student:
2     """
3     Represents a student with a name, age and a list of marks.
4
5     Marks can be passed as separate arguments or as a single iterable:
6     Student("Alice", 20, 80, 90, 85)
7     Student("Bob", 21, [75, 88, 92])
8     """
9
10    def __init__(self, name: str, age: int, *marks):
11        self.name = name
12        self.age = int(age)
13        # Normalize marks: allow either multiple mark args or a single iterable
14        if len(marks) == 1 and hasattr(marks[0], "__iter__"):
15            self.marks = list(marks[0])
16        else:
17            self.marks = list(marks)
18
19    def details(self) -> str:
20        """Return a readable multi-line string describing the student."""
21        marks_str = ", ".join(str(m) for m in self.marks) if self.marks else "No marks"
22        return f"Name: {self.name}\nAge: {self.age}\nMarks: {marks_str}"
23
24    def print_details(self) -> None:
25        """Print the student details to stdout with improved readability."""
26        print(self.details())
27

```

Terminal output:

```

PS C:\Users\VIYSHNAVI\OneDrive\WTMP-LAB> & C:/Python313/python.exe c:/Users/VIYSHNAVI/OneDrive/AI-ASSISTED-CODING/Student.py
PS C:\Users\VIYSHNAVI\OneDrive\WTMP-LAB>

```

OBSERVATION:

The code effectively uses a class to represent a student with attributes for name, age, and marks. It follows good object-oriented practices by encapsulating data and behavior. The use of a list for marks and `sum()` for total calculation makes it scalable and clean. User input integration is smooth, and the docstrings enhance readability. Overall, it's a well-structured and interactive design suitable for basic student data management.

Task Description #4 – Inefficient Loop Refactoring

Task: Refactor this inefficient loop with AI help

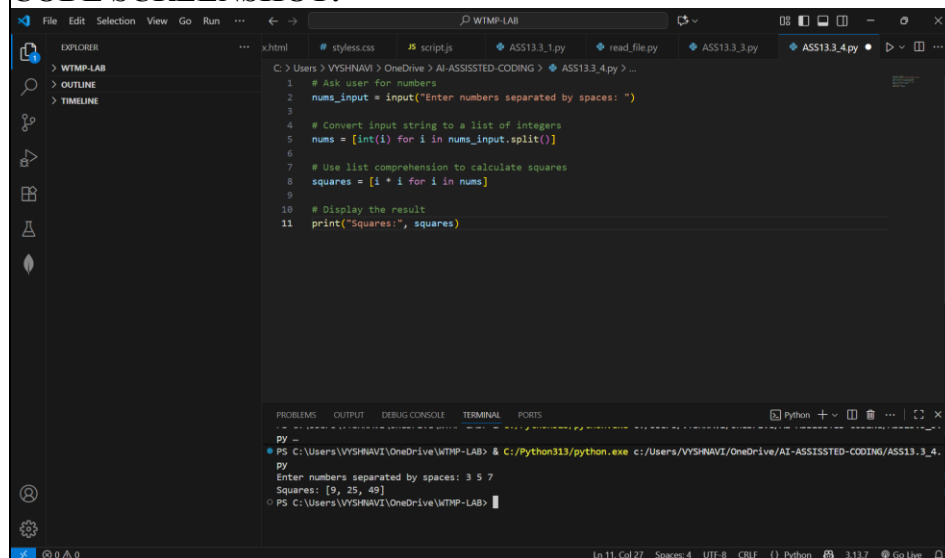
Python Code

```
nums = [1,2,3,4,5,6,7,8,9,10]
squares = []
for i in nums:
    squares.append(i * i)
```

Expected Output: AI suggested a **list comprehension**

PROMPT: Refactor the loop that calculates squares of numbers using list comprehension for improved readability and performance.

CODE SCREENSHOT:

A screenshot of the Visual Studio Code editor interface. The main editor window displays a Python script with the following code:

```
1 # Ask user for numbers
2 nums_input = input("Enter numbers separated by spaces: ")
3
4 # Convert input string to a list of integers
5 nums = [int(i) for i in nums_input.split()]
6
7 # Use list comprehension to calculate squares
8 squares = [i * i for i in nums]
9
10 # Display the result
11 print("Squares:", squares)
```

The script is saved as 'ASS13.3_4.py'. The bottom panel shows the 'TERMINAL' tab with the following output:

```
PS C:\Users\VYSHNAV\OneDrive\WTMP-LAB> & C:/Python313/python.exe c:/Users/VYSHNAV/OneDrive/AI-ASSISTED-CODING/ASS13.3_4.py
Enter numbers separated by spaces: 3 5 7
Squares: [9, 25, 49]
PS C:\Users\VYSHNAV\OneDrive\WTMP-LAB>
```

OBSERVATION:

The updated code swaps out the conventional for loop in favor of a streamlined list comprehension, enhancing clarity and minimizing verbosity. This technique aligns with Pythonic principles, offering a more elegant and efficient solution for straightforward data transformations. It preserves the original logic while making the purpose of the code more transparent. Overall, it's a refined and modern enhancement that reflects best practices in contemporary Python development.